MishMash User Guide

MishMash is a command-line meal prep planner for people who prepare meals in advance and prefer typing commands. Add your recipes, use them to plan meals, and check the ingredients you need to buy.

Start Here: Set up and launch · Find a command · View meals · Check nutrition · View groceries · Record meals · Fix a problem

Release version: The JAR currently published on the Releases page is **v2.0**. The team has verified that this JAR matches the current source code described in this guide. Planned v2.0b features are labelled **Coming soon** below. Data lasts for one running session.

Contents

- Quick Start
- Features
- Frequently Asked Questions
- Known Issues
- Command Summary

Quick Start

Set Up and Launch

1. **Install Java 25.** Use Java 25 for this project.
2. **Download the published v2.0 JAR** from the team's Releases page. A JAR is the file used to run the application. The current build names it `MishMash.jar`.
3. **Run the JAR file from a terminal:**
    - Put the JAR file in an **empty folder**, such as `smoke-test/ip1`. This keeps data files from other applications or earlier tests from interfering with a version that reads or writes files.
    - Open a terminal and **navigate to that folder**. For example, if `smoke-test` is inside your current folder:

      ```
      cd "smoke-test/ip1"
      ```

      Any relative data-file paths are resolved from the terminal's current folder. The current MishMash application keeps its data in memory and does not save it to disk.
    - In the **same terminal**, confirm that Java 25 is selected:

      ```
      java -version
      ```

Check that the version starts with `25`.

- Launch the JAR using `java -jar "{file_name}"`, replacing `{file_name}` with its actual filename. For the current build:

  ```
  java -jar "MishMash.jar"
  ```

  Keep the quotes if the filename contains spaces. Run this command instead of double-clicking the JAR so you can enter commands and read errors in the terminal.

4. **Enter your name** when MishMash asks `What is your name?`, then press Enter. The first line you type is your name, rather than an application command.
5. **Enter one command per line** and press Enter to run it. Enter `bye` to exit.

Try Your First Command

After entering your name, add a recipe:

```
add-recipe n/Chicken and Rice cal/650 p/50 c/70 f/15 i/Chicken Breast:200g i/White Rice:150g i/Olive Oil:10ml
```

MishMash confirms the addition and shows the recipe's nutrition and ingredients. To finish:

```
bye
```

> **Session Data:** Closing or restarting the current application clears the recipes, saved daily targets, meal plan, grocery list, and recorded meals. There is no save or load command.

The Meal Prep Workflow

Use this sequence to generate meals and their groceries:

1. **Add at least three recipes** with `add-recipe`.
2. **Save your daily targets** with `set-targets cal/2000 p/150`, replacing the example numbers with your own. This step is optional; without it, `generate-plan` uses only the targets you type in that command.
3. **Generate meals** with `generate-plan d/3`.
4. **Review meals** with `view-plan` or `view-plan d/1`.
5. **Check daily nutrition** with `check-shortfall cal/2000 p/100`, replacing the example targets with your own. This step is optional.
6. **Generate ingredients** with `generate-list`.
7. **Check your shopping list** with `view-list`.

After generating a replacement plan, generate the grocery list again so the ingredients match the meals you intend to prepare.

To track what you actually eat, use this separate sequence at any time after adding recipes:

1. **Find the recipe** you ate with `list-recipes` and note its number.
2. **Record the meal** with `record-meal 1`, replacing `1` with that number.
3. **Review your intake** with `view-meals`, which shows each recorded meal's nutrition and the totals.

Features

Reading Command Formats

- Enter each command on **one line**. Long examples may wrap visually; do not insert a line break within a command.
- Command words and prefixes are **case-sensitive**. Use `view-plan`, not `VIEW-PLAN`, and `d/`, not `D/`.
- Uppercase words such as `NAME` and `DAY_NUMBER` are placeholders. Replace them with your own values.
- Square brackets indicate optional input. Do not type the brackets. For example, `view-plan [d/DAY_NUMBER]` means either `view-plan` or `view-plan d/1`.
- Separate arguments with spaces. Names may contain spaces without quotation marks, for example `n/Chicken and Rice`.
- For recipe ingredients, use `NAME:QUANTITY_UNIT`, such as `Chicken Breast:200g` or `Olive Oil:2.5ml`. Repeat `i/` for each ingredient.
- `/` is reserved for argument prefixes and is not accepted inside recipe field values.

Prefix or Abbreviation	Meaning
`n/`	Recipe name.
`d/`	Number of days for `generate-plan`; a day number for `view-plan`.
`cal/`, `kcal`	Calories, measured in kilocalories.
`p/`, `P`	Protein, measured in grams.
`c/`, `C`	Carbohydrates, measured in grams.
`f/`, `F`	Fats, measured in grams.
`i/`	Ingredient name, quantity, and unit.
`food/`	Complete recipe food-category declaration; `none` means no supported categories apply.
`diet/`, `avoid/`, `exclude/`	Named diet, avoided categories, and complete ingredient exclusions.
`none`	In `set-targets`, written instead of a number to remove a saved target, for example `cal/none`.

Viewing available commands: `help`

Type `help` to see every supported command, its purpose, and its syntax. This includes `add-recipe`, `list-recipes`, `edit-recipe`, `delete-recipe`, `generate-plan`, `view-plan`, `reset-plan`, `check-shortfall`, `generate-list`, `view-list`, `set-diet`, `view-diet`, `set-targets`, `view-targets`, `record-meal`, `view-meals`, `help`, and `bye`. The `help` command takes no arguments.

A divider and an input prompt appear when the command session starts and after every command, including invalid commands and cancelled actions. Type `bye` to exit.

Adding a Recipe

Purpose: Adds a recipe to your recipe book so you can use it to generate meal plans and grocery lists, or record it as a meal you have eaten.

Format:

```
add-recipe n/DISHNAME cal/CALORIES p/PROTEIN c/CARBS f/FATS i/INGREDIENT:QUANTITYUNIT
[i/INGREDIENT:QUANTITYUNIT ...] [food/TAG[,TAG...]]
```

Before You Start: No recipes or meal plans are required. Have the recipe's calories, protein, carbohydrates, fats, and ingredient quantities ready.

Example: Enter this command on one line after entering your name:

```
add-recipe n/Chicken and Rice cal/650 p/50 c/70 f/15 i/Chicken Breast:200g i/White Rice:150g i/Olive
Oil:10ml food/poultry
```

Result:

```
New recipe added!
Chicken and Rice
650 kcal | Protein: 50g | Carbs: 70g | Fats: 15g
Ingredients:
- Chicken Breast: 200g
- White Rice: 150g
- Olive Oil: 10ml
Food tags: poultry
```

Use `list-recipes` to find the new recipe's number.

Rules:

- Include `n/`, `cal/`, `p/`, `c/`, and `f/` exactly once, and at least one `i/` field. Optional `food/` can appear once. Fields may appear in any order, separated by spaces; prefixes are case-sensitive.
- Recipe and ingredient names must not be blank. Names can contain spaces without quotation marks. `/` is reserved for prefixes and cannot appear in field values; `:` separates an ingredient name from its quantity.
- All four nutrition values must be whole numbers from 0 to 2,147,483,647. Signs and decimal values are rejected. Calories use kcal; protein, carbohydrates, and fats use grams. MishMash uses the nutrition you enter and does not calculate it from the ingredients.
- Each ingredient needs a name, a quantity greater than zero, and a unit. Quantities may be whole numbers or decimals, such as `200g` or `2.5ml`. Write `0.5g`, rather than `.5g`. Units must contain only letters (`A–Z` or `a–z`); there is no fixed list of accepted units. Examples include `g`, `kg`, `ml`, and `cups`. Units are not converted.
- Repeat `i/` for each ingredient. Repeated ingredient names are accepted and remain separate entries in the recipe, in the order entered. Grocery generation later combines entries only when both their names

and units match exactly.

- Duplicate recipe names and identical recipes are accepted as separate recipes. Adding never replaces an existing recipe; use `edit-recipe INDEX` to change one.
- Optional `food/` declares all applicable supported food categories, separated by commas, for example `food/poultry,soy,legume`. Enter `help` for the supported tags. Tags are not inferred from ingredient names. Use `food/none` only when none of the supported categories apply; do not combine `none` with other tags. Omitting `food/` displays `Food tags: Unclassified`, and unclassified recipes are excluded when dietary restrictions are active. `food/none` displays `Food tags: None declared`.
- Adding a recipe does not change existing meal plans, grocery lists, or recorded meals. To use it in a plan, run `generate-plan` again, then `generate-list` for updated groceries.
- Recipes are kept only for the current session and are lost when the application closes.

Errors and Recovery:

The examples below modify the complete example command above; keep its other fields unchanged.

Invalid input or change	Response	What to Do
Omit `f/15`	`Error: Missing required field: f/`	Restore `f/15`. Supply all five required recipe fields.
Leave `n/` blank	`Error: Value must not be blank: n/`	Use `n/Chicken and Rice`. Every supplied prefix needs a value.
Use `p/-5` or `p/2.5`	`Error: Protein must be a non-negative integer.`	Use a whole number such as `p/50`. The same rule applies to all four nutrition fields.
Use `cal/2147483648`	`Error: Calories must not exceed 2147483647.`	Use a value within the supported range, such as `cal/650`.
Remove every `i/` field	`Error: At least one ingredient is required. Use i/INGREDIENT:QUANTITYUNIT, for example i/Rice:150g.`	Restore at least one complete ingredient, such as `i/White Rice:150g`.
Use `i/White Rice:0g`	`Error: Ingredient quantity must be greater than zero.`	Use `i/White Rice:150g` or another positive quantity.
Use `i/White Rice:150` without a unit	`Error: Invalid ingredient format. Use i/INGREDIENT:QUANTITYUNIT, for example i/Chicken Breast:200g.`	Include a unit containing only letters: `i/White Rice:150g`. Negative quantities also fail the ingredient format check.
Use `i/:150g`	`Error: Ingredient name must not be blank.`	Include the name: `i/White Rice:150g`.
Add a second `n/` field	`Error: Repeated field: n/`	Keep only `n/Chicken and Rice`. Only `i/` may be repeated.
Use `protein/50` instead of `p/50`	`Error: Unknown field: protein/`	Use the supported prefix `p/50`.
Use `food/poultry,poultry`	`Error: Repeated food tag: poultry.`	Declare each category once: `food/poultry`. Unsupported tags display an error followed by the valid food tags; choose the applicable supported tags.

Invalid input also displays the `add-recipe` format and an example. No recipe is added until the whole command is valid. Correct the command and enter it again at the next prompt.

Deleting a Recipe

Removes one recipe from your recipe book, chosen by its number. Because a deleted recipe cannot be brought back, MishMash names the recipe and asks you to confirm first.

Before You Start: Add at least one recipe, then use `list-recipes` to find its number.

Format:

```
delete-recipe INDEX
```

Parameter	Meaning
`INDEX`	The number of the recipe to delete, as shown by `list-recipes`. The first recipe is number 1.

Example:

In a fresh session, after entering your name, enter these commands one line at a time:

```
add-recipe n/Chicken and Rice cal/650 p/50 c/70 f/15 i/Chicken Breast:200g i/White Rice:150g i/Olive Oil:10ml
add-recipe n/Protein Oatmeal cal/450 p/35 c/60 f/8 i/Rolled Oats:50g i/Whey Protein:30g
add-recipe n/Salmon Salad cal/550 p/42 c/20 f/32 i/Salmon Fillet:200g i/Mixed Greens:150g
delete-recipe 2
```

MishMash asks you to confirm. Enter `y` to delete the recipe.

Example Output:

In this transcript, `delete-recipe 2` and `y` are the lines you type; the other lines are MishMash's replies.

```
delete-recipe 2
Are you sure you want to delete recipe 2: Protein Oatmeal? (y/n)
y
Recipe deleted!
Protein Oatmeal
Recipes remaining: 2
```

`list-recipes` now shows that the later recipe has moved up one number:

```
Recipes:
1. Chicken and Rice (650 kcal)
2. Salmon Salad (550 kcal)
```

If you answer anything other than `y` or `yes`, nothing is deleted:

```
delete-recipe 2
Are you sure you want to delete recipe 2: Salmon Salad? (y/n)
n
Cancelled. Nothing was changed.
```

Rules:

- Supply exactly one `INDEX`, written as a positive whole number such as `2`.
- Check that the recipe named in the question is the one you meant before you answer. Only `y` or `yes`, in any letter case, confirms. Any other answer, including pressing Enter on its own, cancels the deletion.
- Recipes after the deleted one move up by one number. Run `list-recipes` again before using another number.
- Deleting a recipe does not change an existing meal plan or grocery list. They keep the meals they were generated with, including the deleted recipe. To plan without it, run `generate-plan` again, then `generate-list`. A new plan needs at least three recipes.
- There is no undo. To get a deleted recipe back, add it again with `add-recipe`.

Errors and Recovery:

Situation	Response	What to Do
No recipes have been added	`Error: There are no recipes to delete.`	Add a recipe first.
`delete-recipe 5` with only two recipes	`Error: Recipe index must be between 1 and 2.`	Enter `list-recipes` and choose a number shown.
`delete-recipe`	`Error: Recipe index is required.`	Add a number, for example `delete-recipe 1`.
`delete-recipe 0`	`Error: Recipe index must be at least 1.`	Recipe numbers start at 1.
`delete-recipe abc`, `delete-recipe -1`, `delete-recipe 1.5`, or `delete-recipe 1 2`	`Error: Recipe index must be a single positive whole number.`	Enter one whole number, for example `delete-recipe 2`.
You answer something other than `y` or `yes`	`Cancelled. Nothing was changed.`	Enter `delete-recipe` again and answer `y`.

An index problem is reported before MishMash asks you to confirm, so nothing is deleted. The "no recipes" and "between 1 and N" errors are followed by the list of all command formats. The other errors show only `Format: delete-recipe INDEX`. Enter the corrected command at the next prompt.

Listing Recipes

Shows the recipes in your recipe book. Use it to find a recipe's number, which commands such as `edit-recipe` and `delete-recipe` ask for, or to check one recipe in full. Listing never changes your recipes, meal plan, or grocery list.

Before You Start: Add at least one recipe with `add-recipe`.

Format:

```
list-recipes [INDEX]
```

Command	What You See
`list-recipes`	Every recipe, numbered from 1 in the order you added them, with its calories.

Command	What You See
`list-recipes 2`	The full details of recipe 2: nutrition, ingredients, and food tags.

Example:

In a fresh session, after entering your name, enter these commands one line at a time:

```
add-recipe n/Chicken and Rice cal/650 p/50 c/70 f/15 i/Chicken Breast:200g i/White Rice:150g i/Olive Oil:10ml food/poultry
add-recipe n/Protein Oatmeal cal/450 p/35 c/60 f/8 i/Rolled Oats:50g i/Whey Protein:30g
add-recipe n/Salmon Salad cal/550 p/42 c/20 f/32 i/Salmon Fillet:200g i/Mixed Greens:150g
list-recipes
list-recipes 1
```

Example Output:

`list-recipes` shows:

```
Recipes:
1. Chicken and Rice (650 kcal)
2. Protein Oatmeal (450 kcal)
3. Salmon Salad (550 kcal)
```

`list-recipes 1` shows:

```
Chicken and Rice
650 kcal | Protein: 50g | Carbs: 70g | Fats: 15g
Ingredients:
- Chicken Breast: 200g
- White Rice: 150g
- Olive Oil: 10ml
Food tags: poultry
```

Rules:

- `INDEX` is optional. When given, it must be a single positive whole number, counting from 1.
- The numbers are positions in the current list, and they are the numbers other commands use. They change when a recipe is deleted, so run `list-recipes` again before using a number.
- Calories are in kcal; protein, carbs, and fats are in grams.
- `Food tags` shows `Unclassified` if the recipe was added without `food/`, `None declared` if it was added with `food/none`, and otherwise the declared tags.
- With no recipes, `list-recipes` replies `No recipes yet. Use 'add-recipe' to add one.` This is a normal message, not an error.

Errors and Recovery:

Situation	Response	What to Do
`list-recipes 1` with no recipes	`Error: There are no recipes to show. Use 'add-recipe' to add one.`	Add a recipe, then try again.

Situation	Response	What to Do
`list-recipes 4` with only three recipes	`Error: Recipe index must be between 1 and 3.`	Enter `list-recipes` and choose a number shown.
`list-recipes 0`	`Error: Recipe index must be at least 1.`	Recipe numbers start at 1.
`list-recipes abc`, `list-recipes -1`, `list-recipes 1.5`, or `list-recipes 1 2`	`Error: Recipe index must be a single positive whole number.`	Enter one whole number, for example `list-recipes 2`.

The "no recipes" and "between 1 and N" errors are followed by the list of all command formats. The other errors show only `Format: list-recipes [INDEX]`. Enter the corrected command at the next prompt; errors never change your recipes.

Generating a Meal Plan

If a plan already exists, MishMash asks `Replace the current meal plan and clear its groceries? (y/n)`. Answer `y` or `yes` to generate its replacement. Any other answer cancels and keeps the plan, its targets, and groceries. If generation fails after you agree, the old plan and groceries remain available. See Confirming Plan Changes.

Builds a plan containing three distinct recipes per day, using recipes from your recipe book. Recipes may appear again on different days. Current dietary preferences apply to recipe selection.

Format:

```
generate-plan d/DAYS [cal/DAILY_CALORIES] [p/DAILY_PROTEIN] [c/DAILY_CARBS] [f/DAILY_FATS]
```

Before You Start: Add at least three recipes using `add-recipe`. If dietary restrictions are active, at least three classified recipes must satisfy them.

Example:

```
generate-plan d/2 cal/1700 p/85
```

Result, when matching recipes are available:

```
Meal plan successfully generated for 2 day(s)!
Use 'view-plan' to inspect your planned meals.
```

Rules:

- Request between 1 and 7 days. Each optional daily target must be a positive whole number up to 2147483647.
- Calories use kcal; protein, carbohydrates, and fats use grams. Omitted nutrients have no target unless a target for that nutrient was saved with `set-targets`.
- Without targets, meals are selected randomly. With targets, every day's totals must be within ±15% of every supplied target. For large recipe books, generation searches a bounded sample of combinations.
- Targets you saved with `set-targets` apply to every nutrient you leave out of the command. A target typed here replaces the saved one for this plan only. When a saved target is used, the confirmation adds a

`Daily targets used:` line listing every target that was applied.

- The application retains the targets that were applied. `view-plan` warns whenever a daily total exceeds its exact target, even when that day is within the generation tolerance. These warnings are informational. They keep using the targets the plan was generated with, even if you change the saved targets afterwards.
- Successful generation replaces the active plan and its targets and clears its generated grocery list. A replacement generated without targets has no nutrition warnings. Plans and targets last only for this session.

Example with saved targets: After `set-targets cal/1500 p/80`, a plain `generate-plan d/1` builds a day within 15% of 1500 kcal and 80 g protein, and the confirmation names the targets it used:

```
Meal plan successfully generated for 1 day(s)!
Daily targets used: 1500 kcal, 80 g protein.
Use 'view-plan' to inspect your planned meals.
```

`generate-plan d/1 cal/1400` instead replaces the saved calories with 1400 for that plan and still uses the saved 80 g of protein. It reports `Daily targets used: 1400 kcal, 80 g protein.`

Errors and Recovery: If generation reports that it cannot find three recipes meeting your daily targets within 15%, try different targets or add recipes. If fewer than three eligible recipes are available, add or classify suitable recipes before retrying. Failed generation leaves the previous plan, targets, and grocery list intact. If a saved target was applied, the error adds a line such as `Daily targets applied: 1500 kcal, 200 g protein (including your saved targets). Change those with 'set-targets'.` Use `view-targets` to see what is saved, then change or remove a target with `set-targets`. There is no way to ignore a saved target for a single `generate-plan`; remove it with `none` first.

Generating a Grocery List

Combines the ingredients of every meal in the active plan into one shopping list.

Format:

```
generate-list
```

Before You Start: Add at least three recipes and successfully run `generate-plan d/1` (or choose another supported number of days).

Example:

```
generate-list
```

Result:

```
Grocery list generated. Use 'view-list' to view it.
```

Rules:

- This command takes no arguments and uses every day of the current meal plan.

- Each recipe contributes its ingredients every time it appears, including on different days.
- Quantities are combined only when both ingredient names and units match exactly after trimming. For example, `Rice:0.1g` and `Rice:0.2g` become `Rice:0.3g`, while `Rice:1kg` stays separate. `Rice` and `rice` are also separate. Units are not converted.
- Items follow their first appearance in the meal plan. Generation leaves recipes and planned meals unchanged.
- Running `generate-list` again replaces the stored grocery list instead of accumulating previous totals.
- A successful replacement meal plan clears the old grocery list. A failed plan generation preserves it.
- The list is kept only for the current session. Use `view-list` to inspect it.

Errors and recovery:

Without an active meal plan, the command displays:

```
No active meal plan. Use 'generate-plan' first.
```

Generate a plan and then retry `generate-list`. Unexpected arguments such as `generate-list d/1` display:

```
Error: generate-list does not accept arguments.
Format: generate-list
------------------------------------------------------------
Enter a command, or type 'help' for all commands or 'bye' to exit.
```

Retry without arguments. Invalid syntax does not change an existing grocery list.

Ingredient Combining Example

In a fresh session, enter the following commands after supplying your name:

```
add-recipe n/A cal/500 p/20 c/50 f/10 i/Rice:0.1g
add-recipe n/B cal/500 p/20 c/50 f/10 i/Rice:0.2g
add-recipe n/C cal/500 p/20 c/50 f/10 i/Rice:1kg i/rice:50g
generate-plan d/2
generate-list
view-list
```

There are exactly three recipes, so each appears once per day. The generated list contains these three entries; their display order depends on the order of the meals:

Ingredient	Unit	Quantity for two days	Reason
Rice	g	0.6	`(0.1 + 0.2) × 2`; decimal quantities add exactly.
Rice	kg	2	`1 × 2`; kg is kept separate from g.
rice	g	100	`50 × 2`; lowercase `rice` is a different name from `Rice`.

Running `generate-list` again keeps these totals. It recomputes the ingredients of the current plan, so it does not double the previously generated shopping quantities. Use consistent ingredient names and units when you want entries combined. Nutrition warnings do not prevent grocery generation.

Viewing a Grocery List

Displays the grocery list that has already been generated, including each ingredient's quantity and unit. Viewing does not generate or update the list.

Before You Start: Generate a meal plan and then run `generate-list`.

Format:

```
view-list
```

Example:

```
view-list
```

Example Output:

Use the three `add-recipe` commands and `generate-plan d/2` from the two-day example, then run `generate-list` and `view-list`. You can generate and view groceries without running `check-shortfall`. Each sample meal is used once per day for two days, so the list contains the following quantities. Item order may vary with the meal order.

```
================ Grocery List ================
[ ] Chicken Breast: 400g
[ ] White Rice: 300g
[ ] Olive Oil: 20ml
[ ] Rolled Oats: 100g
[ ] Whey Protein: 60g
[ ] Salmon Fillet: 400g
[ ] Mixed Greens: 300g
==============================================
Total items to purchase: 7
```

Your items and their order depend on the generated list. The number at the bottom counts displayed entries, not the sum of their quantities. For example, `Rice: 1cups` and `Rice: 200g` are two separate entries because their units differ.

Rules:

- `view-list` takes **no arguments**; `view-list d/1` is not supported.
- Quantities retain their units and decimal precision. A quantity such as `2.5ml` is displayed without rounding to a whole number.
- The `[ ]` markers are visual shopping-list markers. There is currently no command to tick or untick them.
- Viewing preserves the list's order and contents. Ingredient combining is done during generation, rather than viewing.

- After generating a replacement meal plan, generate the grocery list again before relying on it for shopping.

Errors and Recovery:

Situation	Response	What to Do
No generated grocery list	`No grocery list generated. Use 'generate-list' first.`	Generate a plan and grocery list, then enter `view-list`.
Extra arguments, such as `view-list extra`	`Error: view-list does not accept arguments.` followed by `Format: view-list`.	Enter `view-list` on its own.

A generated list with no entries displays:

```
================ Grocery List ================
==============================================
Total items to purchase: 0
```

This differs from having no generated list at all, which displays the generate-first message.

Viewing a Meal Plan

Displays the meals in your active plan, with nutrition for each meal and totals for each day. Viewing a plan does not generate new meals or change the plan.

Before You Start: Add at least three recipes, then generate a meal plan using `generate-plan d/2` (or choose another number of days from 1 to 7).

Format:

```
view-plan [d/DAY_NUMBER]
```

Command	What You See
`view-plan`	Every day in the active plan, in day order.
`view-plan d/1`	Only Day 1.
`view-plan d/2`	Only Day 2, if the active plan contains that day.

Example:

```
view-plan d/1
```

Example Output:

```
================ Day 1 ================
Meal 1: Protein Oatmeal (450 kcal | P: 35g, C: 60g, F: 8g)
Meal 2: Chicken and Rice (650 kcal | P: 50g, C: 70g, F: 15g)
Meal 3: Salmon Salad (550 kcal | P: 42g, C: 20g, F: 32g)
-------------------------------------
Daily Totals: 1650 kcal | Protein: 127g | Carbs: 150g | Fats: 55g
=====================================
```

This is an illustrative day. Your meals and their order depend on the generated plan. `view-plan` displays a block like this for each day, separated by a blank line. Meal numbering starts again at 1 for each day.

Daily Nutrition Totals: Each nutrient is added independently for the displayed day, using the full nutrition values entered for its meals. In the example, calories are `450 + 650 + 550 = 1650` kcal, protein is `35 + 50 + 42 = 127` g, carbohydrates are `60 + 70 + 20 = 150` g, and fats are `8 + 15 + 32 = 55` g. Calories are measured in kcal; all three macronutrients are measured in grams. Totals describe planned meals rather than food recorded as eaten. Values are not rounded or adjusted to fit a target, and large daily sums can exceed a single recipe's maximum value of 2147483647.

Rules:

- Day numbers start at **1**, rather than 0. Day 1 is the first day in the plan, not a calendar date.
- Supply at most one `d/` argument, containing a positive whole number.
- If a plan has three days, valid selections are `d/1`, `d/2`, and `d/3`.
- Daily totals add the nutrition of every meal shown for that day; they are not totals for the whole plan.
- After a replacement plan is generated, viewing shows the currently active plan.

Errors and Recovery:

Situation	Response	What to Do
No active plan	`No active meal plan. Use 'generate-plan' first.`	Generate a plan, then enter `view-plan` again.
Day exceeds the plan's length	For a two-day plan: `Error: Day number must be between 1 and 2.`	Enter `view-plan d/1` or `view-plan d/2`.
Zero, negative, decimal, or overflowing day number	A day-number error and `Format: view-plan [d/DAY_NUMBER]`.	Use a positive whole number within the plan's length.
Unexpected or repeated arguments	A format error and `Format: view-plan [d/DAY_NUMBER]`.	Use only `view-plan` or one `d/` argument.

For example, after generating a two-day plan, entering `view-plan d/3` displays:

```
Error: Day number must be between 1 and 2.
Format: view-plan [d/DAY_NUMBER]
------------------------------------------------------------
Enter a command, or type 'help' for all commands or 'bye' to exit.
```

The application remains open. Enter `view-plan d/2` to select a valid day. If there is no active plan, a syntactically valid command such as `view-plan d/2` shows the no-plan message instead.

Nutrition Target Warnings

Each day is compared with the daily targets supplied to `generate-plan`. A total strictly greater than its target produces a warning after the daily totals, before the closing separator. Equal or lower totals and nutrients without a target produce no warning. Warnings appear in calories, protein, carbohydrates, and fats order and show the excess, actual total, and target. Selecting a day displays only that day's warnings.

For example, if the illustrated day above was generated with `generate-plan d/1 cal/1500 p/120`, its daily footer would be:

```
------------------------------------
Daily Totals: 1650 kcal | Protein: 127g | Carbs: 150g | Fats: 55g
Warning: Calories exceed target by 150 kcal (1650 kcal actual; 1500 kcal target).
Warning: Protein exceeds target by 7 g (127 g actual; 120 g target).
====================================
```

Both totals are within the generator's ±15% tolerance but exceed the exact targets. Viewing does not change meals or targets. To use different targets, generate a replacement plan. If `set-diet` repairs a plan, warnings use the repaired meals and the original generation targets; if it clears the plan, no warnings remain. `check-shortfall` uses its own explicit targets and does not change these warnings. There is no separate excess-checking command: use `view-plan` or `view-plan d/DAY_NUMBER` after generating a plan with targets. Generating without targets produces daily totals without warnings.

Setting Dietary Preferences

Purpose: Choose a named diet, avoid food categories, or exclude complete ingredient names. These preferences apply to future plans and can update an existing plan.

Format:

```
set-diet [diet/DIET] [avoid/TAG[,TAG...]] [exclude/INGREDIENT[,INGREDIENT...]]
```

Before You Start: Preferences can be saved before adding recipes. To generate or repair a restricted plan, add at least three compatible recipes with complete `food/` declarations.

Example:

```
set-diet diet/vegan avoid/peanut,soy exclude/mushroom,bell pepper
```

With no active meal plan, the result is:

```
Dietary preferences saved. Future meal plans will use them.
```

Rules:

- Supply at least one field. Each prefix may appear once, in any order.
- Each command **replaces the entire profile**. Omitted fields are cleared. For example, `set-diet avoid/peanut` removes a previously selected vegan diet and any ingredient exclusions. Include every restriction you want to retain.

- Command names and prefixes are case-sensitive. Diet and food-tag values ignore case and surrounding spaces; hyphens and underscores are accepted in multiword values.
- Separate categories and ingredient exclusions with commas. Blank entries and duplicates are rejected, including aliases of the same tag and ingredient names equal after normalization.
- `diet/none` removes the named diet only; any `avoid/` and `exclude/` fields in the same command still apply.
- To clear all restrictions, use `set-diet diet/none` without `avoid/` or `exclude/`. Omitted fields are cleared.
- Preferences and recipe classifications last only for the current session.

Supported Diets and Food Categories

Diet	Excluded categories
`none` (alias: `omnivore`)	No exclusions from a named diet.
`pescatarian`	Beef, pork, poultry, other meat, meat derivatives, and other animal products.
`vegetarian` (alias: `lacto-ovo-vegetarian`)	All pescatarian exclusions, plus fish, crustaceans, molluscs, and gelatin.
`lacto-vegetarian`	All vegetarian exclusions, plus eggs.
`ovo-vegetarian`	All vegetarian exclusions, plus milk and lactose.
`vegan`	All vegetarian exclusions, plus milk, lactose, eggs, and honey.

Supported tags for `food/` and `avoid/` are:

```
milk, lactose, egg, peanut, tree-nut, soy, wheat, gluten,
fish, crustacean, mollusc, sesame, mustard, celery, lupin, sulphite,
beef, pork, poultry, other-meat, meat-derivative, gelatin, honey,
other-animal-product, alcohol, coconut, corn, oat, legume, allium, mushroom
```

Aliases are `dairy` (milk), `soya` (soy), `sulfite` (sulphite), `eggs` (egg), `peanuts` (peanut), `tree-nuts` (tree-nut), `crustaceans` (crustacean), and `molluscs` (mollusc). Use `avoid/peanut,tree-nut` to exclude both nut categories. Categories are matched independently: for example, declare both `milk,lactose` or `wheat,gluten` when both apply, and include `legume` alongside specific legume categories where applicable.

Classifying Recipes

Append the optional `food/TAG[,TAG...]` field when adding a recipe. Declare **every applicable category**, including categories in sauces, stocks, and compound ingredients:

```
add-recipe n/Pancakes cal/400 p/12 c/60 f/10 i/Flour:100g i/Milk:150ml i/Egg:1pc
food/milk,lactose,egg,wheat,gluten
add-recipe n/Rice cal/200 p/4 c/45 f/1 i/Rice:150g food/none
```

`food/none` explicitly declares that no supported categories apply. Omitting `food/` leaves the recipe **unclassified**. Any active restriction excludes unclassified recipes, including an ingredient-only exclusion. `food/none` cannot be combined with tags; `avoid/none` is not supported.

MishMash uses your declarations; it does not identify undeclared allergens or cross-contact. Category names describe food content, not halal or kosher certification. `meat-derivative` covers land-animal derivatives such as stock and lard; also declare the source category where known. Declare both `gelatin` and its source category where known. Use `other-animal-product` for animal-derived ingredients without a more specific category. The vegetarian, vegan, and pescatarian presets exclude that category conservatively.

Ingredient exclusions match **complete names**, ignoring case, surrounding spaces, and repeated spaces. They do not infer synonyms or match substrings: `exclude/chicken` does not exclude `Chicken Breast`; use `avoid/poultry` for the category. Commas separate exclusions and cannot be part of an excluded name.

Updating an Existing Plan

If the new preferences conflict with an active plan, MishMash asks for confirmation before applying them. Answer `yes` to continue; declining preserves the preferences, plan, and groceries. No confirmation is needed when there is no plan or every planned recipe is compatible.

Current state	Result of a valid dietary update
No active plan	Save preferences for future generation.
Every planned recipe is compatible	Save preferences; preserve all days and groceries.
Some days conflict and replacement succeeds	Preserve compatible days in their original positions; replace each conflicting day in full.
A grocery list exists when repair succeeds	Rebuild it from every meal occurrence in the updated plan.
No grocery list exists when repair succeeds	Keep groceries ungenerated; use `generate-list` when needed.
Replacement fails	Save preferences and clear the previous plan and grocery list.

Replacement uses the original daily nutrition targets and the generator's +/-15% tolerance. All three meals in a conflicting day may change. At least three compatible, classified recipes are needed while restrictions are active; having three alone does not guarantee that the nutrition targets can be met. For large recipe pools, generation checks a bounded sample of combinations.

After successful repair with an existing grocery list:

```
Dietary preferences saved. Conflicting days were replaced.
Your grocery list has also been updated.
```

Example: Generate a Restricted Plan

In a fresh session, enter these commands one line at a time after entering your name:

```
add-recipe n/Rice cal/600 p/10 c/20 f/5 i/Rice:200g food/none
add-recipe n/Beans cal/600 p/10 c/20 f/5 i/Beans:200g food/legume
add-recipe n/Tofu cal/600 p/10 c/20 f/5 i/Tofu:200g food/soy,legume
set-diet diet/vegan
generate-plan d/2 cal/1800
view-plan
generate-list
view-list
```

Each day contains these three recipes in an order that may vary. Each day's totals are 1800 kcal, 30 g protein, 60 g carbs, and 15 g fats. The grocery list contains 400 g each of Rice, Beans, and Tofu; item order may vary. The nutrition values are example inputs.

Errors and Recovery:

Input or situation	Response	What to do
`set-diet diet/keto`	`Invalid diet 'keto'.`, followed by valid diets, aliases, and command format.	Choose a supported value, such as `set-diet diet/vegan`.
`set-diet vegan`	Invalid dietary input, supported prefixes, valid diets, and format.	Add a prefix: `set-diet diet/vegan`.
`set-diet`	Missing dietary input and guidance.	Supply at least one field.
`set-diet avoid/milk,dairy`	`Repeated food tag: milk.` and format.	Use `set-diet avoid/milk`.
`set-diet avoid/peanut,`	Blank food-tag error, valid tags, and format.	Remove the trailing comma.
`set-diet diet/vegan diet/vegetarian`	`Repeated field: diet/` and format.	Supply the intended diet once.
`set-diet exclude/Chicken, chicken`	Repeated excluded ingredient error and format.	Supply each normalized name once.
Fewer than three compatible recipes, or no acceptable replacement found	Preferences are saved; a failed-repair explanation is shown and the old plan and groceries are cleared.	Add suitable classified recipes or adjust targets, then use `generate-plan`.

Malformed commands preserve preferences, plans, and groceries. A **valid update with failed repair saves the new preferences and clears generated data**. Check `view-diet` and generate a new plan before viewing groceries again.

Viewing Dietary Preferences

Shows the active dietary profile without changing recipes, preferences, meals, or groceries.

Before You Start: No prerequisites. A fresh session has no restrictions.

Format:

```
view-diet
```

Example: After `set-diet diet/vegan avoid/peanut,soy exclude/mushroom,bell pepper`, enter `view-diet`.

Example Output:

```
Diet: vegan
Avoided food categories: peanut, soy
Excluded ingredients: bell pepper, mushroom
```

Rules: Exclusions appear in sorted order. Empty lists display `none`. The diet also displays `none` in a fresh session. Viewing reads the current profile when executed and never regenerates meals.

Errors and Recovery: `view-diet extra` produces `view-diet does not accept arguments.` followed by `Format: view-diet`. Retry with the command alone; session data is unchanged.

Setting Daily Targets

Saves your daily calorie, protein, carbohydrate, and fat targets for the rest of the session, so you do not have to type them every time you generate a meal plan. Each later `generate-plan` uses the saved targets for any nutrient you leave out of that command. Setting a target never changes a meal plan you have already generated.

Before You Start: None. You can set targets before or after adding recipes.

Format:

```
set-targets [cal/CALORIES|none] [p/PROTEIN|none] [c/CARBS|none] [f/FATS|none]
```

Parameter	Meaning
`cal/CALORIES`	Daily calories in kcal, as a positive whole number.
`p/PROTEIN`	Daily protein in grams, as a positive whole number.
`c/CARBS`	Daily carbohydrates in grams, as a positive whole number.
`f/FATS`	Daily fats in grams, as a positive whole number.
`none`	Written instead of a number, it removes that nutrient's saved target, for example `cal/none`.

Supply at least one parameter, in any order. A nutrient you do not mention keeps its saved target.

Example:

In a fresh session, after entering your name, enter these commands one line at a time:

```
set-targets cal/1500 p/80
set-targets f/37
set-targets p/none
```

Example Output:

```
Daily targets saved: 1500 kcal, 80 g protein.
They apply to the next 'generate-plan'; your current meal plan is unchanged.
```

`set-targets f/37` adds a third target and keeps the first two:

```
Daily targets saved: 1500 kcal, 80 g protein, 37 g fats.
They apply to the next 'generate-plan'; your current meal plan is unchanged.
```

`set-targets p/none` removes the protein target only:

```
Daily targets saved: 1500 kcal, 37 g fats.
They apply to the next 'generate-plan'; your current meal plan is unchanged.
```

Rules:

- Each target must be a positive whole number up to 2147483647. Calories are in kcal; protein, carbohydrates, and fats are in grams.
- `set-targets` changes only the nutrients you name. To replace a target, set it again. To remove one, write `none` (any letter case) as its value. When the last saved target is removed, MishMash replies `All daily targets cleared.`
- Each nutrient may appear only once in a command.
- The next `generate-plan` uses a saved target for every nutrient you leave out of that command. A target typed in `generate-plan` replaces the saved one for that plan only; it does not change what is saved.
- Saved targets do not affect `check-shortfall`, which always uses the targets you type in that command.
- Targets are kept only for the current session. Closing MishMash clears them.

Errors and Recovery:

Situation	Response	What to Do
`set-targets` with nothing after it	`Error: Supply at least one daily target to set or clear.`	Add at least one target, for example `set-targets cal/2000`.
`set-targets cal/0`, `cal/-5`, `cal/abc`, or `cal/1.5`	`Error: Calories target (cal/) must be a positive whole number from 1 to 2147483647, or 'none'.`	Enter a positive whole number, or `none` to remove the target. The message names the nutrient you mistyped.
`set-targets cal/`	`Error: Missing value after cal/`	Add a number or `none` after the slash.
`set-targets x/5` or `set-targets 2000`	`Error: Unrecognized parameter: x/5`	Use only `cal/`, `p/`, `c/`, and `f/`.
`set-targets cal/1 cal/2`	`Error: Parameter cal/ was given more than once.`	Give each nutrient once.

Every error is followed by `Format: set-targets ...` and leaves your saved targets unchanged, because MishMash checks the whole command before saving anything. Enter the corrected command at the next prompt.

Viewing Daily Targets

Shows the daily targets you saved with `set-targets`. Viewing never changes them.

Format:

```
view-targets
```

Example Output:

After `set-targets cal/1500 p/80 f/37`:

```
Daily targets:
Calories: 1500 kcal
Protein: 80 g
Carbs: not set
Fats: 37 g
```

With no targets saved, `view-targets` replies:

```
No daily targets saved. Use 'set-targets' to set them.
```

Rules:

- `view-targets` takes no arguments. `view-targets now` shows `Error: view-targets does not accept arguments.` and `Format: view-targets`.
- A nutrient shown as `not set` has no target, so `generate-plan` does not constrain it unless you type one in that command.

Recording a Meal

Records a recipe as a meal you have eaten, so you can keep track of your actual food intake. Each time you record a meal, MishMash shows its calories and macronutrients, and the totals of everything you have recorded so far. Recording a meal does not change your recipes, meal plan, or grocery list.

Before You Start: Add at least one recipe, then use `list-recipes` to see each recipe's number.

Format:

```
record-meal INDEX
```

Parameter	Meaning
`INDEX`	The number of the recipe you ate, as shown by `list-recipes`. The first recipe is number 1.

Example:

In a fresh session, after entering your name, enter these commands one line at a time:

```
add-recipe n/Chicken and Rice cal/650 p/50 c/70 f/15 i/Chicken Breast:200g i/White Rice:150g i/Olive Oil:10ml
add-recipe n/Protein Oatmeal cal/450 p/35 c/60 f/8 i/Rolled Oats:50g i/Whey Protein:30g
record-meal 1
record-meal 2
```

Example Output:

After `record-meal 1`:

```
Meal recorded!
Chicken and Rice (650 kcal | P: 50g, C: 70g, F: 15g)
Total so far: 650 kcal | Protein: 50g | Carbs: 70g | Fats: 15g
```

After `record-meal 2`:

```
Meal recorded!
Protein Oatmeal (450 kcal | P: 35g, C: 60g, F: 8g)
Total so far: 1100 kcal | Protein: 85g | Carbs: 130g | Fats: 23g
```

Rules:

- Supply exactly **one** index, written as a positive whole number such as `2`. Signs, decimals, and extra words are rejected.
- The index must match a recipe in your current recipe list. If you delete a recipe, the numbers of the recipes after it change, so check `list-recipes` again.
- The meal is recorded with the recipe's calories and macronutrients as stored when you record it. Deleting or editing that recipe later (for example with `edit-recipe`) does not change a meal you have already recorded.
- You can record the same recipe more than once. Each entry counts separately toward the totals.
- Recorded meals are kept only for the current session. They are separate from the meals in a generated plan.
- MishMash does not ask for confirmation. There is currently no command to remove a recorded meal.

Errors and Recovery:

Situation or Input	Response	What to Do
No recipes have been added	`Error: There are no recipes to record. Use 'add-recipe' to add one.`	Add a recipe, then enter `record-meal 1`.
`record-meal`	`Error: Recipe index is required.`	Add an index, for example `record-meal 1`.
`record-meal 5` with only three recipes	`Error: Recipe index must be between 1 and 3.`	Enter `list-recipes` to check the numbers, then use one within that range.
`record-meal 0`	`Error: Recipe index must be at least 1.`	Recipe numbers start at 1, for example `record-meal 1`.
`record-meal two`, `record-meal -1`, or `record-meal 1 2`	`Error: Recipe index must be a single positive whole number.`	Enter one whole number, for example `record-meal 2`.

Errors in the index also display `Format: record-meal INDEX`. The "no recipes" error and the "index too large" error display the list of all command formats instead. Enter the corrected command at the next prompt; errors leave your recorded meals unchanged.

Viewing Recorded Meals

Displays every meal you have recorded, with the calories and macronutrients of each meal, followed by the totals for all recorded meals. Viewing does not record or change anything.

Before You Start: Record at least one meal using `record-meal INDEX`.

Format:

```
view-meals
```

Example:

After the two `record-meal` commands in Recording a Meal, enter:

```
view-meals
```

Example Output:

```
================ Recorded Meals ================
Meal 1: Chicken and Rice (650 kcal | P: 50g, C: 70g, F: 15g)
Meal 2: Protein Oatmeal (450 kcal | P: 35g, C: 60g, F: 8g)
------------------------------------------------
Total: 1100 kcal | Protein: 85g | Carbs: 130g | Fats: 23g
================================================
```

Rules:

- `view-meals` takes **no arguments**.
- Meals appear in the order you recorded them, and are numbered from 1.
- Each meal's line gives its calories (kcal) and its protein (P), carbohydrates (C), and fats (F) in grams.
- The total line adds up all recorded meals. It is not a daily total: MishMash does not record dates, so everything you record in the session is counted together.
- These meals are what you recorded as eaten. They are not the meals in your meal plan, and `check-shortfall` does not use them.

Errors and Recovery:

Situation or Input	Response	What to Do
No meals recorded yet	`No meals recorded yet.`	Enter `record-meal 1` (or another recipe number), then try `view-meals` again.
Extra arguments, such as `view-meals 1`	`Error: view-meals does not accept arguments.` followed by `Format: view-meals`.	Enter `view-meals` on its own.

Checking Nutrition Shortfalls

Use `check-shortfall` to see which days in your active meal plan fall below your supplied daily calorie, protein, carbohydrate, or fat targets, and by how much.

Before You Start: Add at least three recipes, then generate a meal plan. For example, use `generate-plan d/2` to create a two-day plan. You do not need to generate a grocery list before checking nutrition.

Format:

```
check-shortfall [cal/DAILY_CALORIES] [p/DAILY_PROTEIN] [c/DAILY_CARBS] [f/DAILY_FATS]
```

Rules:

- Supply **at least one target**, even though each individual target is optional.
- Targets must be positive whole numbers from **1 to 2147483647**, with calories in kcal and macronutrients in grams. Do not include units in the command.
- Leading zeros are accepted. Signs, decimals, and separators such as commas are rejected.
- Keep each prefix and its value together: write `cal/2000`, not `cal/ 2000`.
- Each prefix can appear once, in any order. Omitted nutrients are not checked.
- The command checks **every day separately** against the same targets. There is no `d/` option; a surplus on one day does not offset another day's shortfall.
- A value equal to or above its target is not reported as a shortfall. Reported nutrients appear in calorie, protein, carbohydrate, then fat order.

Example: Check a Two-Day Plan

In a fresh session, enter these commands one line at a time, after entering your name:

```
add-recipe n/Chicken and Rice cal/650 p/50 c/70 f/15 i/Chicken Breast:200g i/White Rice:150g i/Olive
Oil:10ml
add-recipe n/Protein Oatmeal cal/450 p/35 c/60 f/8 i/Rolled Oats:50g i/Whey Protein:30g
add-recipe n/Salmon Salad cal/550 p/42 c/20 f/32 i/Salmon Fillet:200g i/Mixed Greens:150g
generate-plan d/2
view-plan
check-shortfall cal/2000 p/150 c/200 f/60
```

With exactly these three recipes, each day contains all three meals, although their order may vary. Each day's totals are **1650 kcal, 127 g protein, 150 g carbs, and 55 g fats**. The shortfall check displays:

```
Daily nutrition shortfalls (compared with the supplied targets):
Day 1:
- Calories: 1650/2000 kcal (short by 350 kcal).
- Protein: 127/150 g (short by 23 g).
- Carbs: 150/200 g (short by 50 g).
- Fats: 55/60 g (short by 5 g).
Day 2:
- Calories: 1650/2000 kcal (short by 350 kcal).
- Protein: 127/150 g (short by 23 g).
- Carbs: 150/200 g (short by 50 g).
- Fats: 55/60 g (short by 5 g).
```

Each result shows `planned total/target`, followed by the difference. These targets are example inputs; replace them with the daily values you want to compare against.

Example: Check Only Protein

Using the same plan, enter:

```
check-shortfall p/100
```

The result is:

```
Daily nutrition shortfalls (compared with the supplied targets):
Day 1: No shortfalls for the supplied targets.
Day 2: No shortfalls for the supplied targets.
```

This means the planned protein meets or exceeds 100 g on each day. It does not mean every nutrient meets a target: excesses and omitted nutrients are not checked.

The check reports **any amount below the supplied target**, including amounts within the generator's +/-15% acceptance tolerance. For example, a plan with 1900 kcal may be accepted for a 2000 kcal generation target but will still show a 100 kcal shortfall here.

This command does not change your plan, grocery list, recipes, or generation targets. Supply the targets each time; the check does not save them as personal settings or automatically reuse targets from `generate-plan`. It reports shortfalls without selecting replacement meals. To change the plan, use `generate-plan` again and then repeat the check.

Errors and Recovery:

Situation or Input	Response	What to Do
No active plan, with valid targets	`No active meal plan. Use 'generate-plan' first.`	Add at least three recipes and run `generate-plan d/2`, then retry the check.
`check-shortfall`	`Error: Supply at least one daily nutrition target.`	Add a target, for example `check-shortfall cal/2000`.
`check-shortfall cal/`	`Error: Missing value after cal/.`	Add a value immediately after the slash, for example `check-shortfall cal/2000`. The same rule applies to `p/`, `c/`, and `f/`.
`check-shortfall cal/0`	`Error: Target for cal/ must be a positive whole number from 1 to 2147483647.`	Enter a value within the supported range, for example `check-shortfall cal/2000`. The same rule applies to all four targets.
`check-shortfall p/100 p/120`	`Error: Parameter p/ was given more than once.`	Supply the intended target once: `check-shortfall p/120`.
`check-shortfall d/1`	`Error: Unrecognized parameter: d/1`	Use only `cal/`, `p/`, `c/`, or `f/`, for example `check-shortfall cal/2000`.
`check-shortfall cal/ 2000`	`Error: Missing value after cal/.`	Remove the space after the prefix: `check-shortfall cal/2000`.

Invalid input also displays the command format. Enter the corrected command at the next prompt; errors leave your existing data unchanged.

Editing a Recipe

Update a saved recipe's name, nutrition, ingredients, quantities, or food tags when you change how you prepare it.

Before You Start: Add a recipe, then use `list-recipes` to find its current index. Indexes start at 1. If names are duplicated, choose by index; use `list-recipes INDEX` to check the details.

Format:

```
edit-recipe INDEX [n/NAME] [cal/CALORIES] [p/PROTEIN] [c/CARBS] [f/FATS] [i/INGREDIENT:QUANTITYUNIT
...] [food/TAG[,TAG...]]
```

Supply at least one field. Omitted fields keep their current values. Fields can appear in any order, separated by spaces. Repeat only `i/`; each other prefix can appear once.

Example: In a fresh session, enter these commands after your name:

```
add-recipe n/Chicken Rice cal/650 p/50 c/70 f/15 i/Chicken:200g i/Rice:150g food/poultry
edit-recipe 1 cal/700 p/55 i/Chicken:220g i/Rice:150g
```

The edit produces:

```
Recipe updated!
Chicken Rice
700 kcal | Protein: 55g | Carbs: 70g | Fats: 15g
Ingredients:
- Chicken: 220g
- Rice: 150g
Food tags: poultry
Existing meal plans and grocery lists are unchanged. Generate a new plan and grocery list to use this
edit.
```

For a smaller correction, `edit-recipe 1 p/60` changes only the protein value.

Rules:

- Nutrition values are whole numbers from 0 to 2,147,483,647. Calories use kcal; protein, carbs, and fats use grams. MishMash uses the nutrition you enter and does not calculate it from ingredients.
- If you supply any `i/` fields, they replace the **entire ingredient list**. Include every ingredient you want to keep. To remove an ingredient, omit it from the replacement list. Every ingredient needs a name, a positive whole or decimal quantity, and a unit containing letters, such as `g` or `ml`.
- Names can contain spaces and must not be blank. `/` is reserved for prefixes.
- `food/` replaces the complete food-category declaration. Use the supported tags listed by `help`, such as `poultry`, `milk`, `egg`, or `soy`, separated by commas. Use `food/none` when none apply. Omitted `food/` preserves the old declaration, including an unclassified declaration. When changing ingredients, supply updated tags if the applicable categories change; tags are not inferred from ingredient names.
- Editing preserves recipe indexes and the number of recipes. Existing plans, nutrition shortfall results, and groceries continue to use the recipes chosen when the plan was generated. To use edited recipes, run `generate-plan` again, followed by `generate-list`. Running only `generate-list` still uses the existing plan.
- Edits are held for the current session and are lost when the app closes.

Errors and recovery: Invalid edits leave the recipe unchanged; correct the command and try again.

Input or situation	Message and recovery
No recipes have been added	`There are no recipes to edit.` Add a recipe first.

Input or situation	Message and recovery
`edit-recipe 1`	`Provide at least one recipe field to edit.` Add a field such as `p/55`.
`edit-recipe 0 p/55`	`Recipe index must be at least 1.` Use the latest numbered recipe list.
Index exceeds the recipe count	`Recipe index must be between 1 and N.` Here `N` is the number of recipes.
`edit-recipe 1 p/-5`	`Protein must be a non-negative integer.` Use a whole number such as `p/55`.
`edit-recipe 1 i/Rice:0g`	`Ingredient quantity must be greater than zero.` Supply a positive quantity and the complete replacement ingredient list.
A prefix is blank, unknown, or repeated	The error identifies the invalid field and shows edit usage. Supply a nonblank value and use each prefix once, except repeated `i/`.

Confirming Plan Changes

MishMash asks before these commands can remove meals you already planned:

Command	When it asks	What agreement permits
`reset-plan`	A plan or generated groceries exist	Clear the plan, its targets, and groceries.
`generate-plan d/DAYS ...`	A plan already exists	Replace it and clear its groceries after successful generation.
`set-diet ...`	The current plan conflicts with the new preferences	Replace conflicting days and refresh groceries, or clear the plan and groceries if repair fails.

Only `y` or `yes` confirms, ignoring letter case and surrounding spaces. Any other answer, a blank line, or the end of input cancels with `Cancelled. Nothing was changed.` Cancellation preserves recipes, saved targets, dietary preferences, recorded meals, the plan, and groceries. Enter your next command after the cancellation message. A command typed at the confirmation question counts as the answer; it will not run, so type it again at the next command prompt.

The first plan, a reset with nothing to clear, and dietary changes compatible with the existing meals need no confirmation. Invalid command syntax is rejected before a question appears. Agreement permits an attempt; it does not guarantee generation will find suitable meals.

Cancellation example, with an existing plan:

```
generate-plan d/1
Replace the current meal plan and clear its groceries? (y/n)
n
Cancelled. Nothing was changed.
view-plan
```

The original plan remains available. There is no undo after an agreed change. Closing the application still ends the session and discards its in-memory data.

Resetting a Meal Plan

Use `reset-plan` when you want to start planning again without re-entering recipes or losing your recorded meals.

Format:

```
reset-plan
```

This removes the current meal plan, the targets attached to that plan, and its generated grocery list. Your recipes, dietary preferences (including avoided food categories and excluded ingredients), saved daily targets from `set-targets`, and recorded meals stay available. MishMash asks before clearing an existing plan or groceries. Answer `y` or `yes` to proceed; a confirmed reset cannot be undone.

Example:

```
generate-plan d/2 cal/1650
generate-list
reset-plan
Reset the current meal plan and clear its groceries? (y/n)
yes
```

The first two commands require enough suitable recipes. After the reset, MishMash displays:

```
Meal plan reset. Recipes, dietary preferences, and recorded meals are unchanged.
```

`view-plan` and `check-shortfall` will ask you to generate a plan first. `view-list` will ask you to generate groceries first. You can still use `list-recipes`, `view-diet`, and `view-meals` to see the data you kept. To start again, run `generate-plan d/1`, then `generate-list`. Your saved daily targets still apply to nutrients you leave out of the generation command. Targets supplied only for the removed plan are not reused; supply those again if needed. Use `view-targets` to check your saved targets.

If there is no current plan or groceries, no confirmation is needed. MishMash displays `No active meal plan to reset.` and lets you continue. Repeating the reset is safe. The command accepts no arguments: `reset-plan d/1` produces `reset-plan does not accept arguments.` followed by `Format: reset-plan`, and keeps the current plan unchanged.

Upcoming v2.0b Features: Coming soon

These features are planned under the v2.0b milestone and are **not implemented in this checkout**. No command syntax is available yet.

Feature	Status	Tracking issue
Replace one planned meal with another recipe	**Coming soon (v2.0b)**	#57
Adjust the portion of one planned meal	**Coming soon (v2.0b)**	#58
Change ingredients for one planned meal without changing its recipe	**Coming soon (v2.0b)**	#60

Feature	Status	Tracking issue
Review recorded meals by date	**Coming soon (v2.0b)**	#62
Correct or remove a recorded meal	**Coming soon (v2.0b)**	#63

Other v2.0b stories already have support in this checkout, including resetting a plan, confirming plan changes, and viewing dietary preferences. Their existing instructions describe that support. Persistence is not labelled Coming soon here because issue #51's intended scope still needs clarification.

Exiting MishMash

Enter:

```
bye
```

MishMash displays `Goodbye!` and closes. In the current application, all session data is cleared when the application exits.

Frequently Asked Questions

Do I need to read the whole guide to get started?

No. Follow Quick Start, then use Command Summary to jump to the task you need.

Will my recipes still be here next time?

No. The current application keeps data only for the running session. Restarting starts a fresh session.

Does viewing generate new meals or ingredients?

No. `view-plan` and `view-list` display existing data. Use the corresponding generation command first.

Can I tick items off the grocery list?

The current `[ ]` markers are display-only. No shopping-item marking command is available.

What is the difference between my meal plan and my recorded meals?

The meal plan, created by `generate-plan`, is what you intend to eat. Recorded meals, created by `record-meal`, are what you have actually eaten. They are separate: recording a meal does not change the plan, and `view-meals` does not show planned meals.

Can I remove a meal I recorded by mistake?

Not yet. There is currently no command to remove or edit a recorded meal. Restarting MishMash clears all recorded meals.

Known Issues

- **Session-only data:** Recipes, dietary settings, daily targets, plans, groceries, and recorded meals are cleared on exit. There is no save, load, or data-transfer command.
- **Meal-log limitations (Coming soon in v2.0b):** Recorded meals have no dates or editable serving quantities. Individual entries cannot be corrected or removed.
- **Planning limitations (Coming soon in v2.0b):** There is no command to replace one planned meal, scale its portion, or change only its ingredients. Edit the source recipe and regenerate the plan and groceries when needed; existing snapshots keep their earlier values.
- **Grocery limitations:** Units are not converted and `[ ]` markers cannot be ticked off.

See Troubleshooting for recovery steps.

Troubleshooting

Problem	What to Check
The terminal does not recognise `java`	Install Java 25 and ensure the terminal can find it. Open a new terminal, then run `java -version`.
Java reports an incompatible class version	Run `java -version` in the same terminal used to launch the JAR, and select Java 25.
`Unable to access jarfile`	Check the filename and the terminal's current folder. Put quotes around filenames or paths containing spaces.
The application greets me instead of running my first command	Enter your name first. The next line is your first command.
`Error: Unknown command: ...`	Use the spelling and lowercase command word from the command table, or enter `help`.
`Error: Enter a command.`	Enter a supported command instead of a blank line.
A plan or grocery list is missing	Run `generate-plan d/2` after adding at least three recipes. Run `generate-list` before `view-list`, including after replacing a plan.
A generated plan still shows a nutrition shortfall	Generation accepts daily totals within +/-15% of supplied targets; `check-shortfall` reports any amount below its supplied targets. See Checking Nutrition Shortfalls.
A generated plan shows an excess warning	A daily total is above an exact generation target, even if it is within the accepted 15% tolerance. See Nutrition Target Warnings.
Ingredients that look similar appear separately	`generate-list` combines exact ingredient names and units only. Check capitalization and units; see the ingredient combining example.
`record-meal` reports an index error	Run `list-recipes` and use a number shown there. Numbers change after `delete-recipe`. See Recording a Meal.
`view-meals` shows `No meals recorded yet.`	Record a meal first with `record-meal INDEX`. Meals in a generated plan are not recorded automatically.
`delete-recipe` answers `Cancelled. Nothing was changed.`	Only `y` or `yes` confirms. Enter the command again and answer `y`. See Deleting a Recipe.

Problem	What to Check
A deleted recipe still appears in my meal plan or grocery list	Existing plans and lists keep the meals they were generated with. Run `generate-plan`, then `generate-list`, to build new ones.

Invalid viewing input displays an error or a generate-first message; you can enter another command without restarting.

Recovering from Invalid Plan or Recipe Arguments

For `generate-plan`, `list-recipes`, and `delete-recipe`, invalid arguments include the relevant `Format:` line with the error. For example:

```
generate-plan d/0
```

produces:

```
Error: DAYS must be a whole number from 1 to 7.
Format: generate-plan d/DAYS [cal/DAILY_CALORIES] [p/DAILY_PROTEIN] [c/DAILY_CARBS] [f/DAILY_FATS]
------------------------------------------------------------
Enter a command, or type 'help' for all commands or 'bye' to exit.
```

Correct the value and retry, for example `generate-plan d/1` after adding enough eligible recipes. This invalid request does not change your existing plan or grocery list. If reporting a problem, include the command, displayed error, and relevant diagnostic log; the application retains the underlying validation cause for troubleshooting.

Command Summary

Click a task to jump directly to its instructions. The commands below are supported by the current team version.

I Want To…	Enter	Before Using It
Add a recipe	`add-recipe n/NAME cal/CALORIES p/PROTEIN c/CARBS f/FATS i/INGREDIENT:QUANTITY_UNIT [food/TAG[,TAG...]]`	Include all fields and at least one ingredient.
List recipes	`list-recipes [INDEX]`	Omit the index to list all recipes; supply one to see its full details.
Edit a recipe	`edit-recipe INDEX p/55`	Use an index from the latest recipe list and supply at least one field.
Delete a recipe	`delete-recipe INDEX`	Use an index from the latest recipe list, then confirm the deletion.
Set dietary preferences	`set-diet diet/vegetarian avoid/peanut`	Supply at least one field; this replaces the complete profile.
View dietary preferences	`view-diet`	No prerequisites.
Set daily targets	`set-targets cal/2000 p/150`	Supply at least one target; use `none` to remove one.

I Want To…	Enter	Before Using It
View daily targets	`view-targets`	No prerequisites.
Generate meals	`generate-plan d/3`	Add at least three recipes; choose 1 to 7 days.
Reset my meal plan	`reset-plan`	Clears the current plan and groceries; keeps your other session data.
View every day	`view-plan`	Generate a meal plan first.
View one day	`view-plan d/1`	Choose a day in the active plan.
Check nutrition shortfalls	`check-shortfall cal/2000 p/100`	Generate a plan first; supply at least one daily target.
Generate groceries	`generate-list`	Generate a meal plan first.
View groceries	`view-list`	Generate a grocery list first.
Record a meal I ate	`record-meal INDEX`	Add at least one recipe; use an index from the latest recipe list.
View recorded meals	`view-meals`	Record at least one meal to see nutrition totals.
Show command help	`help`	No prerequisites.
Exit	`bye`	Session data will be cleared.